

INSTITUTO TECNOLÓGICO DE COSTA RICA
Campus Tecnológico Central Cartago
Escuela de Ingeniería en Computación

IC3101 – Arquitectura de Computadoras
II Semestre 2024

Reporte #9 - Semana 10 – Sivarama - Capitulo 15
Ejecución condicional

Estudiante: Mauricio González Prendas – **Carné:** 2024143009

Profesor: Esteban Arias Méndez

Fecha de entrega: 30/09/2024

Bibliografía:

[1] S. P. Dandamudi, *Guide to Assembly Language Programming in Linux*. Springer Science & Business Media, 2005, pp. 317–339.

Abstract—*The assembly language offers various instructions for handling conditional execution, such as unconditional jumps, comparisons, conditional jumps, and loops. While assembly lacks high-level control structures like if-then-else and loops found in modern programming languages, it provides basic instructions to implement these constructs. Unconditional jumps transfer control directly or indirectly to a target address. Conditional jumps test specific flags after a comparison, and loop instructions manage iterations by decrementing a counter. Indirect jumps use registers or memory to specify the destination, making them useful for multiway branching like switch statements. Through these instructions, assembly can implement complex control flows.*

I. Introducción

El lenguaje ensamblador no proporciona directamente estructuras de decisión como las de los lenguajes de alto nivel, tales como las estructuras de selección if-then-else o los bucles while y for. Sin embargo, ofrece instrucciones básicas que permiten construir estas estructuras, como las instrucciones de salto incondicional, comparaciones, salto condicional y bucles. En esta sección, se explora cómo estas instrucciones permiten la ejecución condicional en ensamblador.

II. Saltos incondicionales

El salto incondicional se realiza con la instrucción `jmp`, que transfiere el control a una dirección objetivo especificada. Esta dirección puede ser dada de manera **directa** o **indirecta**.

A. Saltos directos

En un salto directo, la dirección objetivo está incluida directamente en la instrucción. Un ejemplo de salto directo es:

```
jmp etiqueta
```

Donde etiqueta es una referencia a la instrucción destino. Este tipo de salto puede ser **adelante** (cuando la instrucción de destino está después del jmp) o **atrás** (si está antes).

B. Saltos relativos

Los saltos directos usan direcciones **relativas**. La dirección de destino se calcula como un desplazamiento respecto a la siguiente instrucción tras el jmp. Esto permite que el código sea **relocatable**, útil para el código independiente de posición.

C. Saltos intersegmento

En un salto intersegmento, tanto el registro de segmento (CS) como el registro de puntero de instrucción (EIP) se actualizan, permitiendo saltar a otro segmento de memoria.

III. Instrucción de comparación

La instrucción cmp realiza una comparación aritmética entre dos operandos y ajusta las banderas de estado (flags) sin modificar los operandos. Los resultados de la comparación se utilizan en conjunto con las instrucciones de salto condicional para tomar decisiones.

Un ejemplo de comparación es:

```
cmp AL, DL
```

La operación ajusta varias banderas, incluidas **CF** (acarreo), **ZF** (cero), **SF** (signo), y **OF** (desbordamiento), que luego determinan el flujo del programa mediante saltos condicionales.

IV. Saltos condicionales

Las instrucciones de salto condicional permiten ejecutar saltos basados en el valor de ciertas banderas. Estas instrucciones se dividen en tres grupos:

A. Saltos basados en banderas individuales

Estos saltos ocurren cuando una bandera específica (por ejemplo, ZF o CF) se encuentra en un estado particular. Se ofrecen dos versiones por bandera: una para cuando la bandera está activa (por ejemplo, jz para salto si ZF está activa) y otra para cuando no lo está.

B. Comparaciones sin signo

Cuando se comparan valores sin signo, las banderas **CF** y **ZF** se utilizan para evaluar las relaciones como mayor, menor, igual, etc. Instrucciones como **ja** (saltar si mayor) y **jb** (saltar si menor) se usan para estos casos.

C. Comparaciones con signo

Para valores con signo, se utilizan las banderas **SF**, **ZF** y **OF**. Por ejemplo, la instrucción **jg** salta si un número es mayor, mientras que **jl** lo hace si es menor.

V. Instrucciones de bucle

El registro **CX** o **ECX** se utiliza para contar repeticiones en las instrucciones de bucle como **loop**, **loope** y **loopne**. Estas instrucciones decrementan el valor de **CX** y saltan a una etiqueta si el valor no ha alcanzado cero. Son útiles para crear bucles controlados por contadores en ensamblador.

Ejemplo de bucle:

```
mov CX, 5 ; Inicia el contador en 5
repeat:
    ; código del bucle
    loop repeat ; Decrementa CX y salta si no es 0
```

VI. Saltos indirectos

Los saltos indirectos utilizan registros o memoria para determinar la dirección de destino. A diferencia de los saltos directos, aquí la dirección de destino no está codificada directamente en la instrucción. Por ejemplo, la instrucción:

```
jmp [ECX]
```

Transfiere el control a la dirección almacenada en **ECX**. Este tipo de saltos es útil para implementar construcciones como las sentencias **switch** o tablas de salto en ensamblador.

VII. Uso de saltos condicionales en tablas de salto

Una estructura común en lenguajes de alto nivel es el uso de sentencias **switch** para controlar múltiples condiciones. En ensamblador, este tipo de lógica se puede implementar mediante una combinación de comparaciones y saltos condicionales. Una tabla de salto puede ser creada en memoria, donde cada entrada corresponde a una dirección de código para ejecutar una acción basada en una condición.